Devoir 1

Titre : Système de Gestion et d'Audit d'une Station de Recherche Arctique

Objectif :
Vous êtes l'ingénieur logiciel d'une station de recherche isolée en Arctique. Vous devez créer un programme en C++ pour gérer l'inventaire des ressources critiques (nourriture, carburant, oxygène) et enregistrer les incidents techniques. Une ressource ici est caractérisée par un nom, sa quantité, son type (Vital, technique, consommable) et l'information spécifique sur soit sa température ou sa pression. Votre programme doit être robuste : une erreur non gérée dans ces conditions extrêmes pourrait être fatale !

Instructions & Fonctionnalités :

1. **Chargement des Données** : Au démarrage du programme il serait judicieux pour vous d'effectuer un chargement de données pour avoir accès aux informations des ressources en mémoire, lisez le fichier "database1.txt" pour remplir un tableau de structures représentant les ressources.
2. **Interface Menu** : Créez un menu interactif.

```
--- STATION ARCTIQUE : MENU DE GESTION ---
1. Afficher l'etat des ressources
2. Consommer une ressource
3. Trier par quantite (Croissant)
4. Quitter
Votre choix : 
```

- 1. **Afficher l'état des ressources** : qui affiche les ressources en mémoire, contenue dans le tableau.

```
NOM      QTE      TYPE     SPEC
Oxygene 85        Pres: 1013
Fuel     1200     Temp: -25.5
Rations 45        Pres: 0
Batteries         30       Temp: -10.2
Medicaments       15       Pres: 1010
```

- 2. **Consommer une ressource** : qui demande à l'utilisateur l'index de la ressource à consommer (0 à 4), la quantité à consommer et fera les opérations nécessaires.

```
--- STATION ARCTIQUE : MENU DE GESTION ---
1. Afficher l'etat des ressources
2. Consommer une ressource
3. Trier par quantite (Croissant)
4. Quitter
Votre choix : 2
Index de la ressource (0-4) : 1
Quantite a consommer : 10
Mise a jour effectuee.
```

- 3. **Trier par quantité** : cette option permet de trier les ressources par taille.

```
NOM      QTE      TYPE     SPEC
Medicaments       15       Pres: 1010
Batteries         30       Temp: -10.2
Rations 45        Pres: 0
Oxygene 85        Pres: 1013
Fuel     1200     Temp: -25.5
```

- 4. **Quitter** : pour sortir du programme.

```
--- STATION ARCTIQUE : MENU DE GESTION ---
1. Afficher l'etat des ressources
2. Consommer une ressource
3. Trier par quantite (Croissant)
4. Quitter
Votre choix : 4
Fermeture du systeme...
```

L'interface (le programme) doit s'arrêter uniquement quand l'utilisateur choisira de quitter.

3. **Journalisation (Logs)** : Chaque opération comme **Afficher l'état des ressources** (Succès ou erreur + message), **Trier par quantité** (Message : "inventaire trie par quantité"), **la consommation d'une ressource et lorsque l'utilisateur choisir une option qui n'est pas parmi les 4** (Par exemple 5), devra être **sauvegarder en mémoire dans un fichier** log appelle **"journal_bord.txt".**

journal_bord.txt

```
[LOG] Chargement des donnees reussi.
[LOG] Consommation de 1 unites de Batteries
[LOG] Chargement des donnees reussi.
[LOG] Chargement des donnees reussi.
[LOG] Chargement des donnees reussi.
[LOG] Chargement des donnees reussi.
[LOG] Inventaire trie par quantite.
[LOG] Chargement des donnees reussi.
[LOG] Consommation de 13 unites de Medicaments
[LOG] Consommation de 10 unites de Fuel
[LOG] Chargement des donnees reussi.
```

4. **Gestion des Ressources** :
    - Utilisez une **énumération** pour définir le type de ressource (VITAL, TECHNIQUE, CONSOMMABLE).
    - Utilisez une **union** à l'intérieur de la structure pour stocker soit une température (double), soit un niveau de pression (int) selon le type.
5. **Algorithme de Tri** : Implémentez un algorithme de tri (au choix) pour classer les ressources par quantité restante.
6. **Gestion des Erreurs** : Utilisez des blocs try/catch pour gérer le chargement de données, les saisies utilisateurs invalides et les tentatives de consommer plus de ressources que le stock disponible.

L'étudiant pourra concevoir les fonctions suivantes :

- **`void chargerDonnees(...)`** : Prend en paramètre un tableau de structures (via pointeur) et le nom du fichier. Doit gérer l'ouverture du fichier.
- **`void trierRessources(...)`** : Reçoit le tableau. L'étudiant peut choisir un *Tri à bulles* (simple) ou un *QuickSort* (s'il veut une meilleure note de complexité).
- **`void mettreAJourStock(...)`** : Utilise des pointeurs pour modifier la ressource choisie. Doit inclure un throw si la quantité devient négative.
- **`void enregistrerLog(string message)`** : Ouvre le fichier en mode ios::app pour ne pas écraser les anciennes données.

Contraintes Techniques :

- **Entête du fichier .cpp** : Votre code doit impérativement commencer par votre Nom&Prenom, NI, espace pour votre note et commentaire, voici un exemple :

```
//####################################
//# Nom : Maurel Kouekam
//# NI : A0022****
//# Note de code :    /15
//#
//# Lien Video Explicative: https://www.youtube.com/watch?v=g0L-ZLEg4hA
//#
//# Note Video:  /5
//#
//# Commentaire du prof :
//#
//#
//####################################
```

- **Nommage des structures** : Vous **DEVEZ** ajouter vos initiales à la fin de chaque nom de structure (*ex : struct RessourceMK pour Maurel Kouekam*), uniquement les structures dont vous aurez besoin pour votre projet.

Livrables :

1. Un seul fichier **.cpp** propre, indenté et commenté nommé **nom_prenom.cpp**.
2. Une vidéo (lien de la video) de **5 minutes maximum** qui présentera :
   a. **Votre solution, les tests et la sauvegarde dans le fichier**
   b. Expliquez votre choix d'algorithme de tri et sa complexité
   c. Expliquez comment vous passez vos structures en paramètres (pointeurs ou références) et pourquoi cela impacte la performance.

Rappel théorique (Fichiers) :

- Pour lire : `ifstream fichier("nom.txt"); while(fichier >> var1 >> var2)...`
- Pour écrire : `ofstream logs("logs.txt", ios::app); logs << message;`

[Vous pourrez effectuer des recherches complémentaires ;-)]

Données d'Entrée (database1.txt**) :**

J'ai joint un fichier "database1.txt" (que vous pourrez télécharger dans le répertoire de votre fichier .cpp) contenant des données de base qui pourront être utilisées pour le remplissage du tableau de structures (Ressource).

database1.txt

```
Oxygene 85 0 1013
Fuel 1200 1 -25.5
Rations 45 2 0
Batteries 30 1 -10.2
Medicaments 15 0 1010
```

Pour la vidéo :

L'idéale pour moi serait que vous créez la vidéo de maximum 5 minutes sur votre PC puis vous l'uploadez en ligne sur n'importe quelle plateforme, par exemple vidéo en ligne non répertoriée de YouTube.
J'aimerai juste que vous partagiez le **lien de ladite vidéo directement dans votre code**, dans la section commentaire comme indique en image tout en haut.

Ici je joins les liens de ressources intéressantes pour faire tout cela, sinon faites des recherches sur comment faire :

- Enregistrer vidéo sur Windows : https://www.youtube.com/watch?v=1o9EAFNv0o4
- Enregistrer vidéo sur Mac : https://www.youtube.com/watch?v=cT9ClauUsHQ
- Ajouter une vidéo dans votre répertoire privée YouTube : **https://www.youtube.com/watch?v=g0L-ZLEg4hA**

Barème de Correction :

Code (/15)

- Concepts appropriés (/5) : Utilisation correcte des structures imbriquées, unions, énumérations et exceptions.
- Manipulation Mémoire (/3) : Usage pertinent des pointeurs ou références dans les fonctions pour optimiser les appels.
- Clarté et Indentation (/2) : Respect du style, indentation logique et présence de l'entête requis.
- Commentaires (/2) : Explication de la logique des blocs complexes (non triviaux).
- Exécution et Robustesse (/3) : Le programme compile, le menu fonctionne et il ne plante pas si l'utilisateur saisit une mauvaise valeur.

Vidéo (/5)

- Qualité de l'explication (/3) : L'étudiant démontre qu'il comprend ce qui a été demande et la solution qu'il propose.
- Analyse de Complexité (/2) : Justification correcte de l'impact de son approche sur les ressources de la machine.

Cout << « Bonne Chance ☺ << endl ;